

졸업 프로젝트 5월 1차 격주보고서		
SNN을 활용한 고효율 추천 시스템 개발 및 최적화 연구		
2020058259 이승현 2020061676 이강혁		지도교수 김상욱 교수님
지난 2주간 진행사항	SNN 기반 추천 시스템의 구조 설계를 위한 데이터셋 설정 및 암시적 피드백 인코딩 및 입력 형태 구체화	O
	SNN 추천 시스템 관련 논문 더 찾아보고 이해하기	O
	전력 비교를 수행한 연구 조사 및 실험 설계 방향 정립	△
	Neural CF layer - GMF와 MLP 구현	O
진행 내용	1. SNN 기반 추천 시스템의 구조 설계를 위한 데이터셋 설정 및 암시적 피드백 인코딩 및 입력 형태 구체화 (1) 구조 설계 기존의 협업 필터링 및 딥러닝 기반 추천 시스템은 명시적 피드백에 크게 의존하는 반면, 실제 서비스 환경에서는 클릭, 조회, 찜, 구매와 같은 암시적 피드백이 더 빈번하게 사용되고, 또한 사용자의 무의식적 행동을 잘 반영한다는 점에서 암시적 피드백 기반 추천 시스템의 필요성이 커지고 있습니다. (2) 데이터셋 설정 암시적 피드백 기반 SNN 추천 시스템을 구현하기 위해서는 사용자-아이템 상호작용 데이터가 체계적으로 구성되어야 합니다. 주요 데이터 필드로는 사용자 식별자(user_id), 아이템 식별자(item_id), 상호작용 유형, 타임스탬프가 필수적으로 포함됩니다.	

이때 상호작용 유형은 클릭, 조회, 찜, 구매 등으로 구분되며, 각 이벤트 유형별로 중요도 가중치를 일반적으로 다음과 같이 부여합니다. (구매=1.0, 찜=0.7, 조회=0.4, 클릭=0.1)

시간 정보의 경우 절대적인 타임스탬프보다는 첫 이벤트를 $t=0$ 으로 한 상대적 시간(Δt)으로 변환하는 것이 효과적이며, 사용자 활동의 자연스러운 구분을 위해 30분 이상 비활성 시 새로운 세션으로 분할하는 전처리가 필요합니다.

(3) 암시적 피드백 인코딩 기법

SNN에 입력하기 위해 암시적 피드백을 spike train으로 변환하는 주요 기법으로는 네 가지 접근 방식이 있습니다.

1) Time Window Encoding

- 고정된 시간 간격으로 이진 값(0/1) 기록
- 단순 구현이 가능하지만 시간 해상도가 낮음

2) Rate Coding

- 이벤트 중요도에 따라 스파이크 빈도를 조절
- Poisson 과정 기반 인코딩
- ex)
 - 구매 → 10Hz
 - 찜 → 7Hz
 - 조회 → 4Hz
 - 클릭 → 2Hz

3) Latency Coding

- 이벤트 중요도에 따라 스파이크 발생 시간을 달리 설정
- 중요할수록 빠른 시간에 스파이크 발생
- ex)
 - 구매 → 2ms
 - 찜 → 5ms
 - 조회 → 8ms
 - 클릭 → 10ms

4) Population Coding

- 이벤트 유형별로 별도의 뉴런 채널을 할당
- ex)
 - 클릭 → 뉴런 1
 - 조회 → 뉴런 2
 - 찜 → 뉴런 3
 - 구매 → 뉴런 4

(4) 입력 데이터 표현

- Spike Tuple 기반 입력

(neuron_id, spike_time) 형태의 튜플 리스트로 데이터를 구성할 예정입니다. 각 뉴런은 상호작용 유형에 해당하며, spike_time은 상대적인 시간 정보 또는 latency coding 방식에 따라 설정할 계획입니다. 예를 들어,

[(1, 10.0), (2, 8.0), (4, 2.0)]는 뉴런 1(클릭)에서 10ms, 뉴런 2(조회)에서 8ms, 뉴런 4(구매)에서 2ms에 스파이크가 발생했음을 의미합니다.

『데이터셋』

(1) AliExpress User Behavior Dataset (Alibaba)

행동 유형:

- 클릭
- 장바구니 추가
- 구매
- 상품 조회

(2) Taobao User Behavior Dataset (Alibaba)

행동 유형:

- pv (page view, 조회)
- buy (구매)
- cart (장바구니 추가)
- fav (찜하기)

2. 전력 비교를 수행한 연구 조사 및 실험 설계 방향 정립

- 전력 비교에서의 주요 기준

(1) MAC (Multiply and Accumulate), AC (Accumulate) 개수

(2) 메모리 접근 횟수 (SNN에서는 spike의 sparsity와 관련이 있습니다)

보통 SNN이 이산 데이터 형식을 사용하여 MAC가 아닌 AC만 사용하기 때문에 전력 효율이 ANN보다 높다고 알려져 있습니다. 하지만, 실제로 에너지 효율을 결정 짓는 데에 memory access (sparsity, data reuse)가 더 중요하므로 이를 종합적으로 고려해야 합니다.

그 이유에는,

- SNN은 AC만 사용된다는 장점, data reuse가 힘들다는 단점
- ANN은 data reuse가 용이하다는 장점, MAC가 많이 사용된다는 단점

과 같은 특징이 있기 때문입니다. 계산량만 보았을 때는 단순히 SNN을 사용하는 것만으로 전력 효율성이 높아진다고 할 수 있지만, SNN은 data reuse가 불가능하기에 모델, 데이터 종류 등에 따라 memory access의 차이가 생기게 됩니다. 그렇기에 memory access로 인한 전력의 증가가 AC만이 사용되어 감소한 전력 차이보다 커진다면, 전력에 있어서 효율적이라고 할 수 없게 됩니다.

참고한 논문에서는 이를 꼬집으며 memory access를 고려한 전력 비교 방식을 제안하고 있습니다.

이는 SNN과 ANN의 비교에서 동일한 이벤트 기반 아키텍처를 가정하고, 하드웨어가 아닌 디지털 환경에서의 비교를 전제로 합니다.

SNN의 한 layer에서 총 에너지(결과값)은 layer에 존재하는 시냅스에 input으로 들어온 모든 spike의 개수와 뉴런에 input으로 들어온 spike를 timestep에 따라 가중하여 계산한 전위 값의 합입니다. 일반적으로 시냅스 연산 에너지가 timestep 업데이트 에너지에 비해 크기 때문에 각 timestep에서의 업데이트 에너지는 시냅스 연산 에너지에 비해 무시할 수 있게 됩니다. 이 경우 모든 SNN 모델은 유사한 에너지 소비량(IF+inst와 유사)을 가지며 시냅스 당 평균 spike만이 ANN과 비교하여 에너지

효율성을 결정합니다. 물론, timestep마다 업데이트되는 spike, 상태 변수 등의 값을 저장하기 위해 메모리 접근이 일어나므로 timesteps의 크기가 큰 경우에는 이를 고려하여야 합니다. 일반적으로 (timesteps= 10)에서 뉴런 업데이트와 관련된 오버헤드를 무시할 수 있고, (timesteps = 500)에서는 중요한 고려 사항이 됩니다.

timesteps가 충분히 작은 경우, 각 timestep에서 업데이트되는 SNN 모델의 상대적인 에너지 효율은 ANN과 비교하여 뉴런 개수에 대한 시냅스 개수의 비율, spike sparsity에 따라 달라집니다. 이 비율이 0.5를 넘으면 SNN은 Eyeriss와 같은 효율적인 ANN보다 전력 효율이 떨어집니다. 그러나 비율이 감소함에 따라 SNN의 에너지 효율성은 빠르게 증가합니다.

이를 통해 이론적으로 spike sparsity를 구하여 SNN의 상대적인 전력 효율을 비교할 수 있음을 알 수 있었습니다.

참고 문헌

<https://ieeexplore.ieee.org/abstract/document/9927729>

<https://www.sciencedirect.com/science/article/pii/S1383762122002508>

<https://ieeexplore.ieee.org/abstract/document/9340948>

-전력 비교 방식 조사

neuromorphic chip 하드웨어를 직접 사용하지 않고, neuromorphic chip 환경을 제공하는 시뮬레이터를 사용할 예정입니다.

아래는 neuromorphic chip을 사용한 시뮬레이터를 제공하는 기관입니다.

(1) EBRAINS : <https://www.ebrains.eu/data/find-data/>

(2) BindsNET : <https://bindsnet-docs.readthedocs.io/>

(3) NEST : <https://www.nest-simulator.org/>

3. Neural CF layer - GMF와 MLP 구현 시 고려사항 (진행중)

GMF와 MLP의 학습 불균형

GMF는 얇은 구조로 되어 있어 학습 시 경사하강법이 빠르게 적용되고, 수렴 속도 또한 빠른 편입니다. 반면 MLP는 상대적으로 깊은 구조를 가지기 때문에 학습 속도가 느리고, 초기에는 충분히 표현을 학습하지 못한 상태로 남게 되는 경우가 많습니다. 이로 인해 학습이 진행되는 동안 GMF의 출력이 과도하게 모델 예측에 영향을 미치고, MLP는 실질적인 기여를 거의 하지 못하는 문제가 발생합니다. 두 경로 간 학습 속도와 수렴 시점이 다르면 전체 모델 성능에도 악영향을 주게 됩니다.

이러한 문제를 해결하기 위한 일반적인 방법으로는 두 경로의 출력을 단순히 합치거나 연결하는 것이 아니라, 일정한 가중치를 두고 조합하는 방식이 자주 사용됩니다. 예를 들어 GMF 출력에 α , MLP 출력에 $(1 - \alpha)$ 를 곱하여 두 출력을 조합하는 방식입니다. 이때 α 는 하이퍼파라미터로서, 두 경로의 상대적인 영향력을 조정하는 역할을 합니다. 학습 초기에는 GMF의 수렴 속도가 빠르므로 α 값을 작게 설정하여 GMF의 영향력을 줄이고, MLP가 충분히 표현을 학습할 수 있도록 하는 전략이 효과적입니다.

또 다른 접근으로는 두 출력값의 크기 차이를 조정하여 균형을 맞추는 방식이 있습니다. GMF와 MLP는 서로 다른 구조이기 때문에 출력의 값 범위나 분포가 다를 수 있으며, 이로 인해 결합 시 한쪽이 일방적으로 결과에 영향을 주기도 합니다. 이를 방지하기 위해 출력값에 정규화를 적용하거나, 학습 가능한 보정 계층을 추가하여 두 출력의 스케일을 자동으로 조정하는 방법이 활용됩니다. 예를 들어, 각 출력에 정규화 계층을 적용하거나, 선형 계층을 통해 자동으로 크기를 조정하는 방식입니다.

하지만 위와 같은 방법이 최선의 결과를 이끌어 낼 수 있는지 확실하지 않습니다. 실제 적용 환경에 따라 성능 차이가 발생할 수도 있기 때문에 추가로 학습하고 실험해 보며 안정적인 결합 방식을 찾을 필요가 있다고 생각합니다.

진행 중 이슈	SNN의 성능과 높은 전력 효율성을 위해서 전용 하드웨어인 neuromorphic chip이 필요합니다. 다만, 현재 neuromorphic chip의 상용 판매는 이루어지고 있지 않고, 관련 기업 혹은 기관에 승인 요청을 제출하여 승인을 받아야만 사용이 가능합니다. 이를 위해 승인 요청을 보낼 예정입니다.
다음 4주간 계획	Neural CF layer - GMF와 MLP 통합 단계까지 구현 완료 및 학습을 통한 효율성 검증